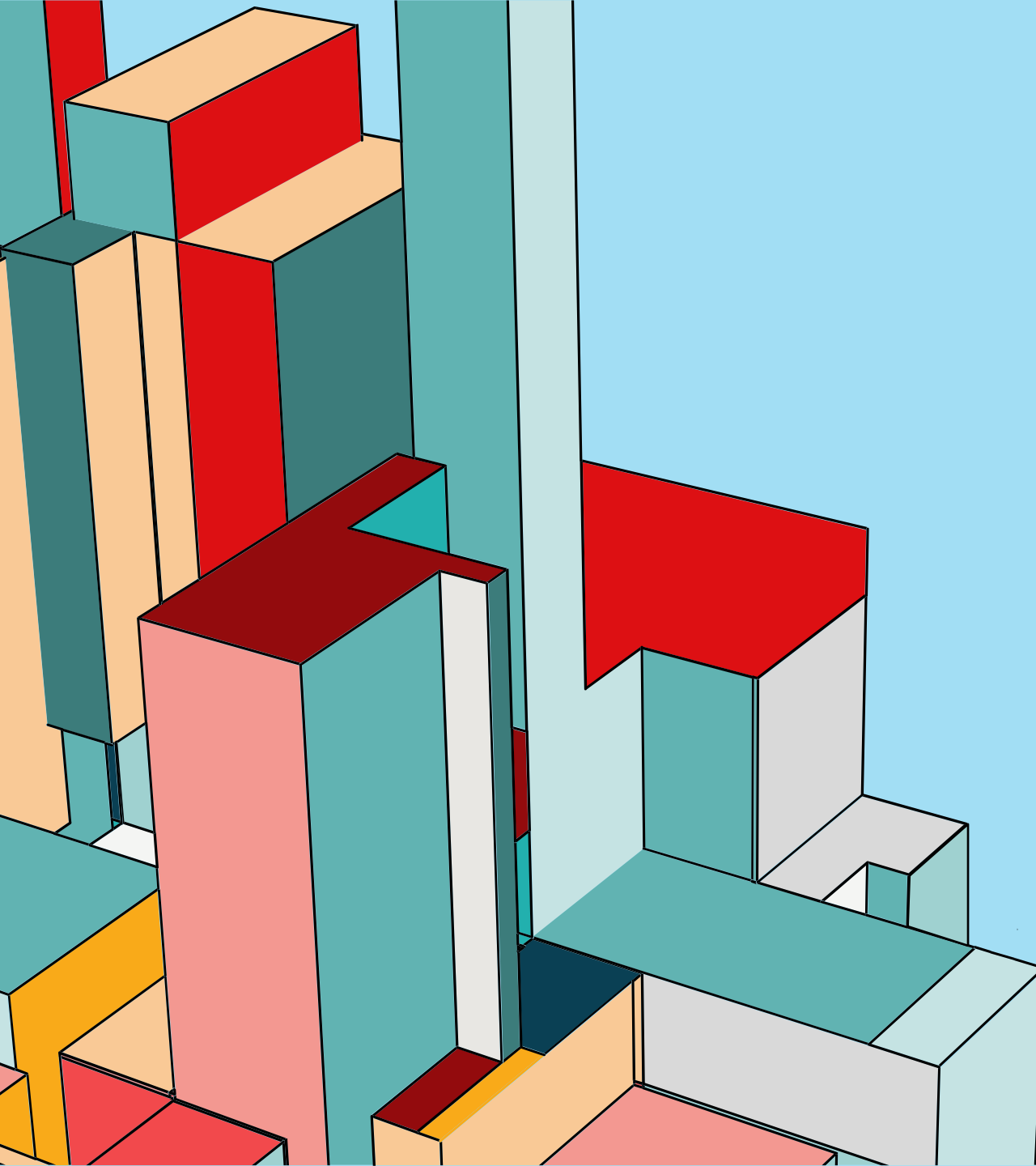
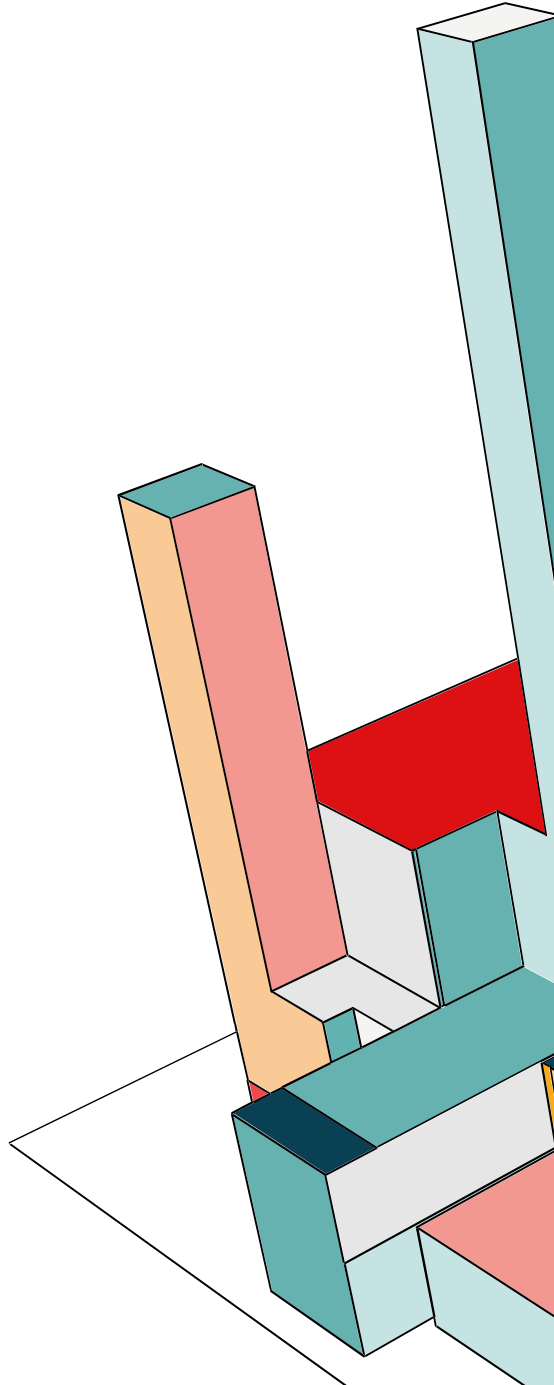




CSIS-2300 INTRODUCTION TO ALGORITHMS

TODAY'S AGENDA

- Course motivation
- Drivers of pedagogy
- Your instructional team
- Course structure & administrative details
- Initial course content





**“COMPUTER SCIENCE IS AS
MUCH ABOUT COMPUTERS
AS ASTRONOMY IS ABOUT
TELESCOPES.”**

--Edsger Dijkstra (maybe)

THREE PARADIGMS OF COMPUTER SCIENCE

- "Computer science is a branch of physics & electrical engineering. Every computation involves a change in the state of a physical object, and so the device used is everything."
- "Computer science is all about communicating ideas and processes precisely and efficiently. The devices matter to the extent that they promote or inhibit the flow of information between people."
- "Computer science is an abstract, mathematical discipline. It is fundamentally about what it is possible to compute, and what of that can be done efficiently. The devices used are irrelevant."

CAOS

PSoft

FOCS &
Algo



YOU ARE HERE



**“GET INTO A RUT EARLY:
DO THE SAME PROCESSES
THE SAME WAY.
ACCUMULATE IDIOMS.
STANDARDIZE.”**

--Alan Perlis (first Turing Award
winner), “Epigrams on
Programming”



Photo credit: ACM

GOAL #1: LEARN THE COMMON “IDIOMS”

- Many problems in computer science are well-understood, with known good solutions.
- Furthermore, these problems tend to appear over and over again in many different contexts.
- Thus, every professional in the field is expected to recognize these situations and be able to quickly implement these solutions.
 - And no, that doesn't mean “vibe coding” your way to an answer – how would you know it was correct?

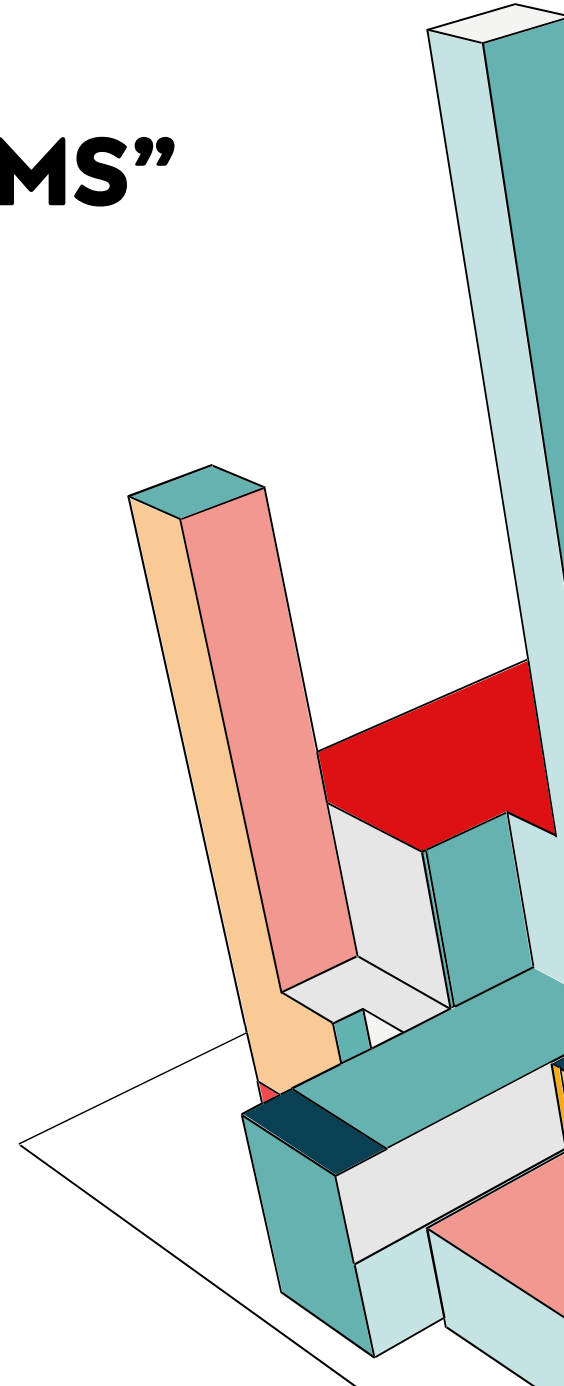
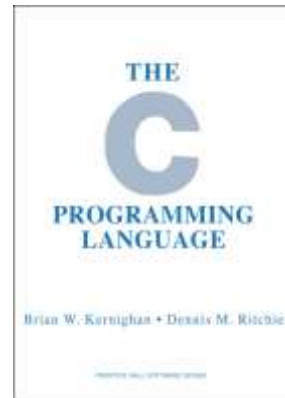


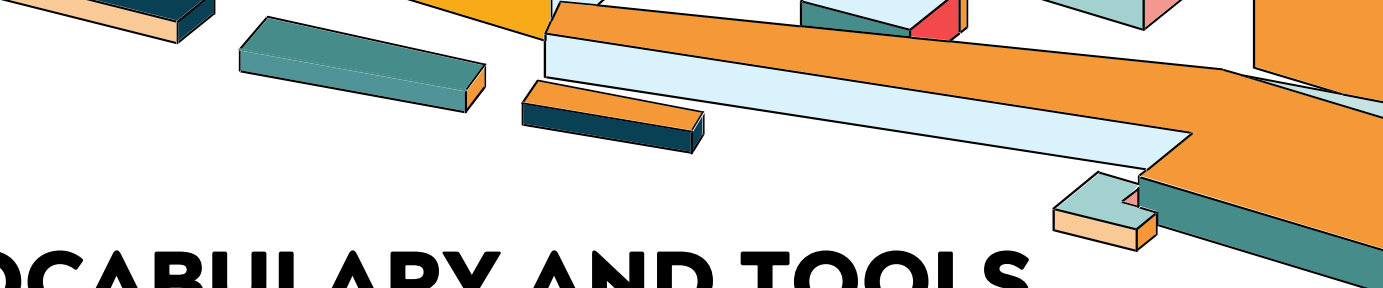


Photo credit: Princeton University

**“CONTROLLING
COMPLEXITY IS THE
ESSENCE OF COMPUTER
PROGRAMMING.”**



--Brian Kernighan
(co-author of *the* textbook on C)



GOAL #2: DEVELOP VOCABULARY AND TOOLS FOR ANALYZING COMPLEXITY

- You know big-O notation...
- ... but do you really? Can you prove that an algorithm runs in, for example, $O(n^2)$ time? What, precisely, does that mean?
- There are also other complexity notions besides big-O; the most common one we'll look at is big- Θ .
- We want to be able to rigorously, mathematically define these complexity classes and then prove when an algorithm belongs to one of them.

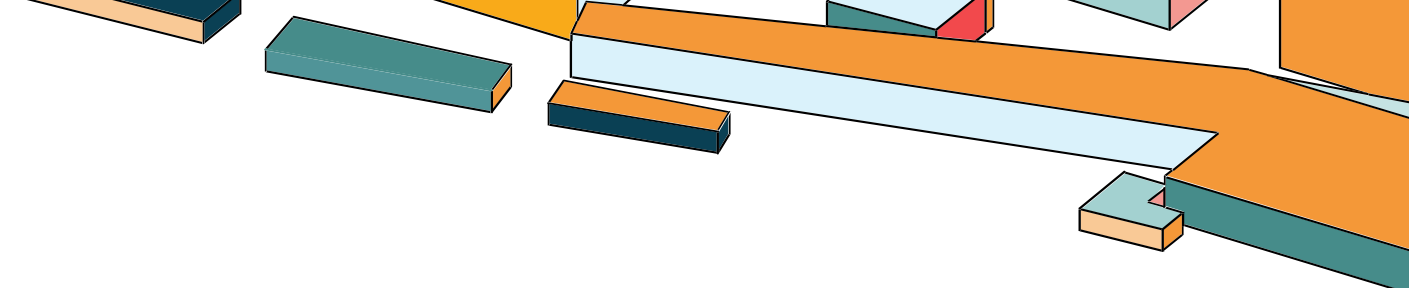


Daguerreotype by Antoine Claudet, 1843

**“ONE ESSENTIAL OBJECT IS
TO CHOOSE THAT
ARRANGEMENT WHICH
SHALL TEND TO REDUCE TO
A MINIMUM THE TIME
NECESSARY FOR
COMPLETING THE
CALCULATION.”**

--Augusta Ada King,
Countess of Lovelace





GOAL #3: THE RIGHT TOOL FOR THE RIGHT JOB

- Most problems have more than one correct solution (as in, they will eventually produce the correct answer).
- However, there's often a demonstrably best answer – one that runs fastest, or in the least amount of memory, or that meets some other constraint.
- Recognizing (or even defining) the problem and its requirements is a necessary part of selecting an algorithm.

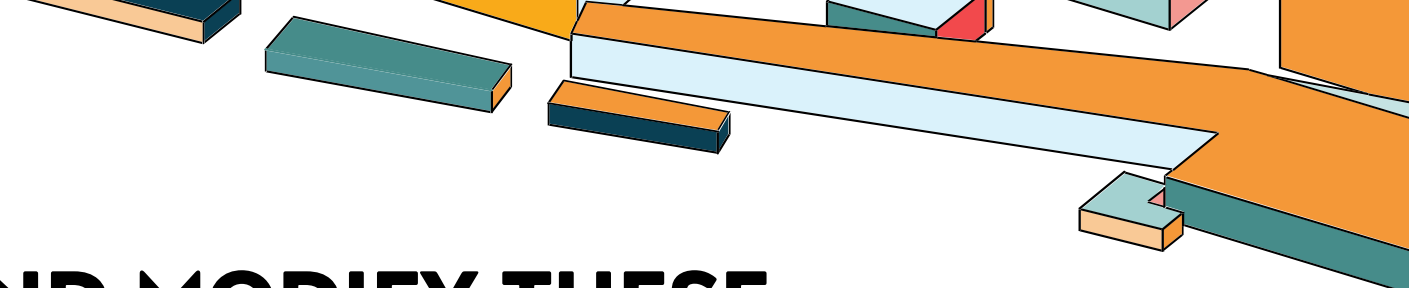
**“THE KEY TO
UNDERSTANDING
COMPLICATED THINGS IS
KNOWING WHAT NOT TO
LOOK AT.”**

--Gerald Jay Sussman



Still image from recorded
lecture at MIT





GOAL #4: COMBINE AND MODIFY THESE ALGORITHMS TO SOLVE NEW PROBLEMS

- Ultimately, you would like to be able to use the techniques from this class to synthesize solutions to new problems.
- The space of possible solutions is uncountably infinite – you want to be able to eliminate categories of solutions so that you can quickly settle on the solution archetype that is most likely to be useful.



Matters of Pedagogy

“ATTENTION IS AN ACHIEVEMENT.”

--JAMES LANG, KANEB CENTER FOR TEACHING EXCELLENCE, NOTRE DAME

Paying attention to one thing for hours at a time is not our default state – humans are hard-wired to be distractible.

Our phones, etc. are precisely designed to take advantage of this.

Focusing and retaining information is challenging – it requires intent, dedicated practice, and a supportive environment.

One of the key promoters of attention is what everyone else is doing! When the folks around you are on task, it is easier for you to continue to focus as well.

PROMOTING COMMUNAL ATTENTION

In general, devices off and put away.

Tablets/laptops for notes are OK... but please stick to notes.

(Also, you won't have your devices for exams – I strongly urge you to consider paper notes.)

If you're going to have extended text conversations, please step into the hallway until you're done.

I will build a dedicated break (~5 minutes) into the middle of each lecture.

Please try to arrive on time – interruptions once we have started break momentum.

TWO ANECDOTES FROM MACHINE LEARNING

<https://youtu.be/tlOlHko8ySg>

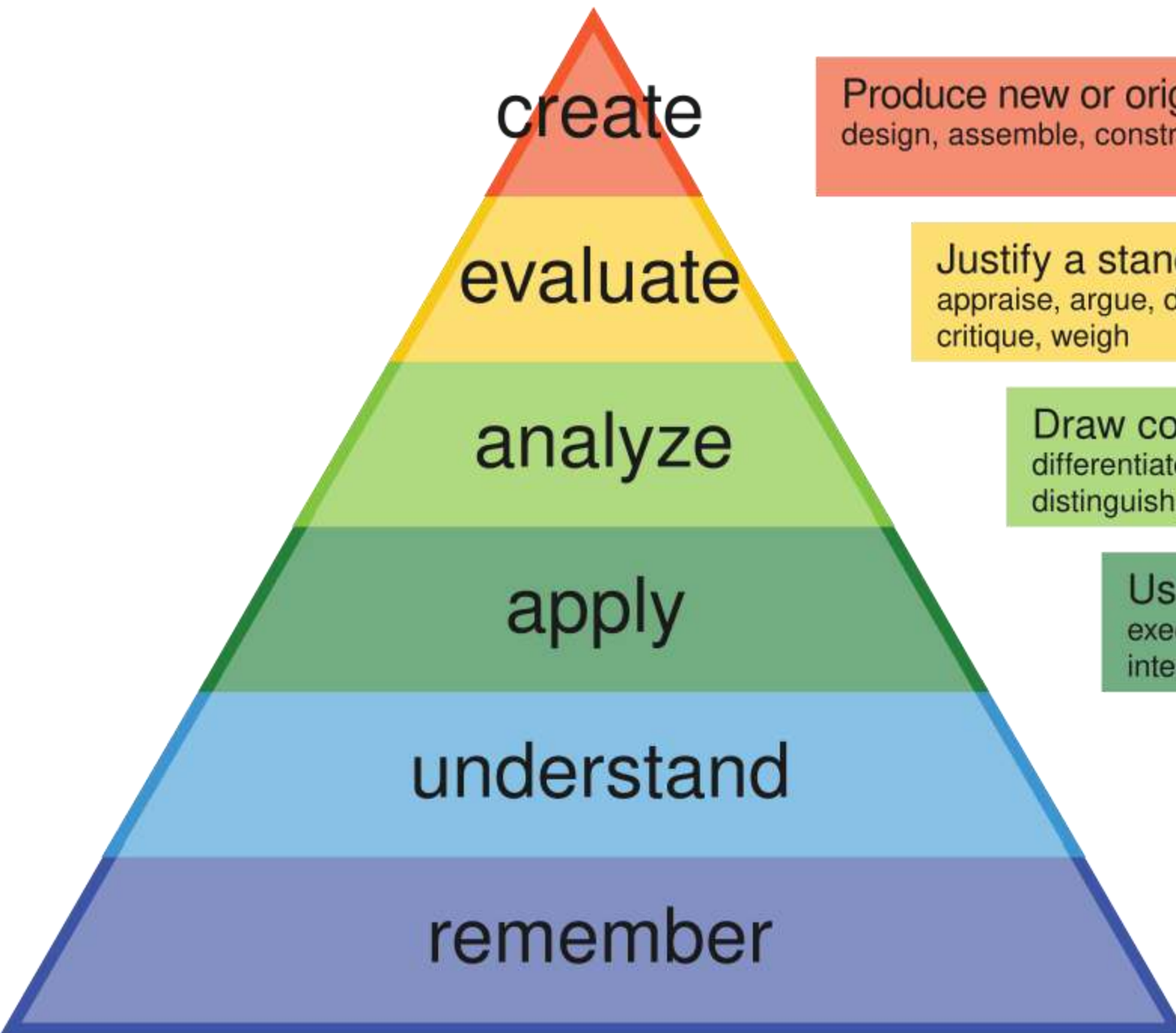
[https://www.ted.com/talks/peter haas the real reason to be afraid of artificial intelligence](https://www.ted.com/talks/peter_haas_the_real_reason_to_be_afraid_of_artificial_intelligence) (start at 2:29)

A SURPRISING STATEMENT

Correct answers are
NOT the goal!

GOODHART'S LAW (I.E. GAMIFICATION)

- When a measure becomes a target, it ceases to be a good measure.
- Homework problems, test scores, grades in general... All of these things are supposed to be measures, not targets.
- If your goal is "Get an A in Algorithms", you're already missing the point!



create

Produce new or original work

design, assemble, construct, develop, formulate, investigate, write, present

evaluate

Justify a stand or decision

appraise, argue, defend, judge, select, support, value, critique, weigh

analyze

Draw connections among ideas

differentiate, organize, relate, compare, contrast, distinguish, examine, experiment, question, test

apply

Use information in new situations

execute, implement, solve, use, demonstrate, interpret, operate, schedule, sketch

understand

Explain ideas or concepts

classify, describe, discuss, explain, identify, locate, recognize, report, select, translate

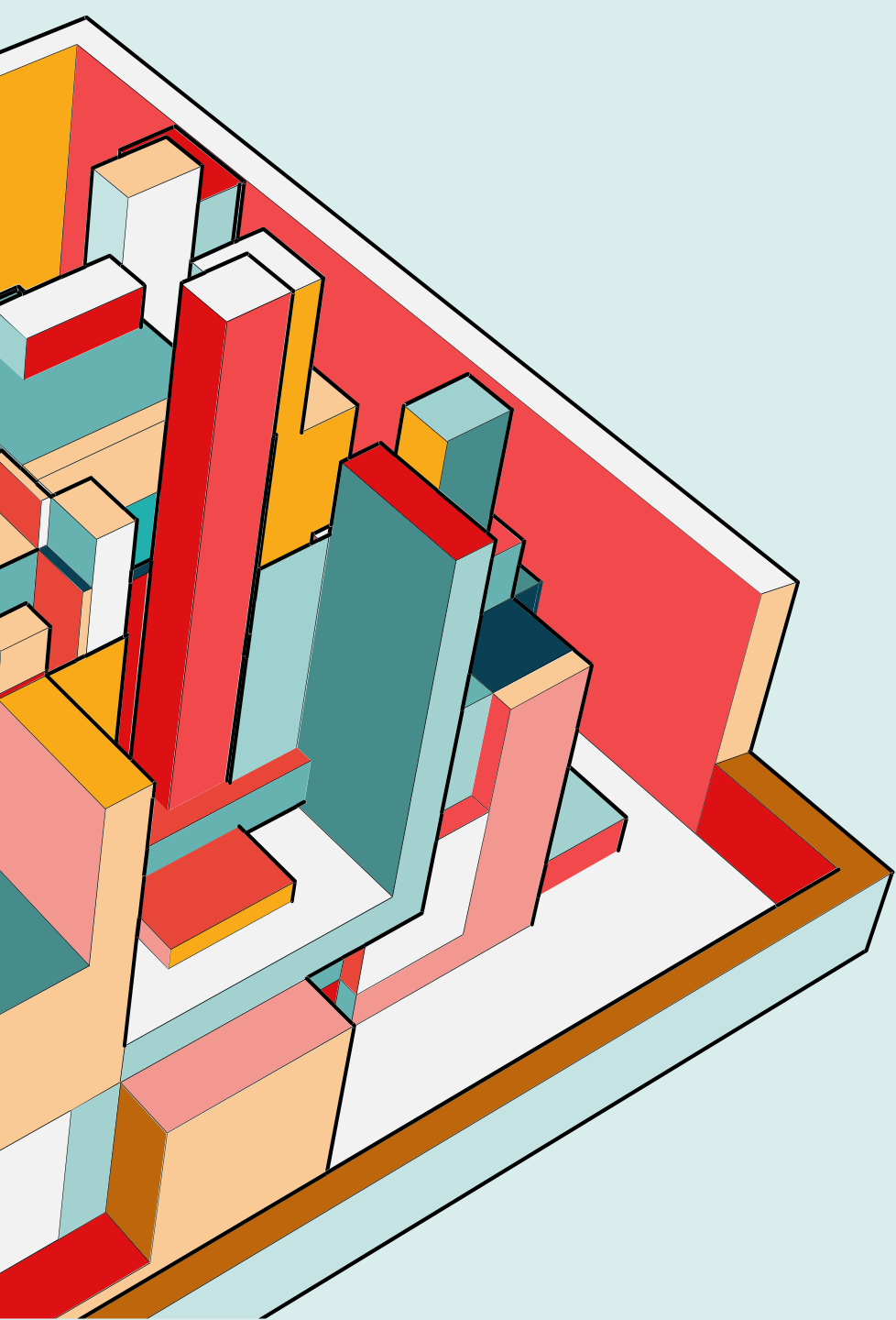
remember

Recall facts and basic concepts

define, duplicate, list, memorize, repeat, state

THE TRUE GOAL

- You want to be able to use the content from this course outside of this classroom. No one is going to care if you got problem 3 on homework 7 correct after May.
- OTOH, if you can analyze a problem, evaluate how various algorithms will perform in solving that problem, and use the tools from this class to create a solution and prove that your solution works – THAT is what we're striving for.
 - Questions during lecture, homework, exams – all of these are merely the clunky tools that we have to measure this understanding.

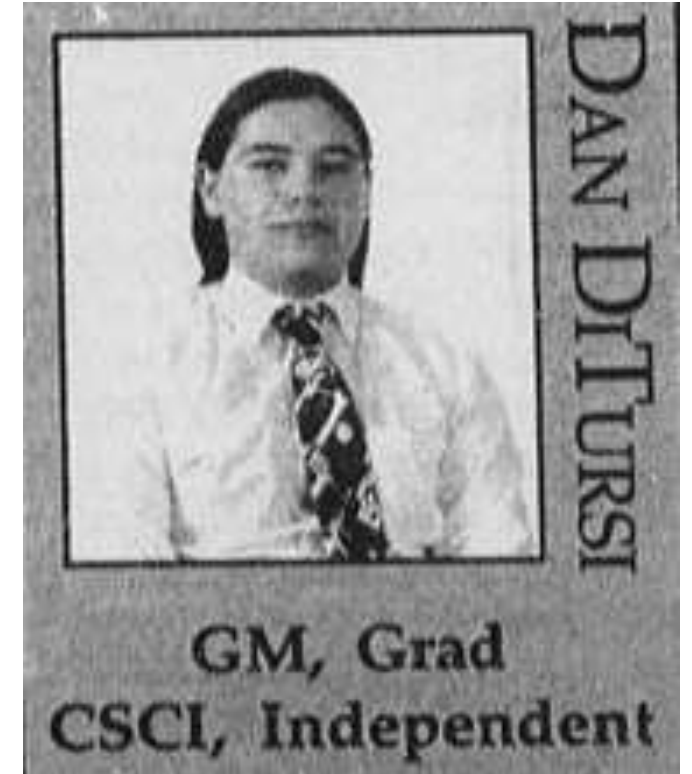


INSTRUCTIONAL TEAM

DR. DAN DITURSI '98



- Office: AE 123A (down the steps, first office on the right)
- E-mail: ditursi@rpi.edu
- Office hours:
 - Tuesdays 12:30-14:30
 - Thursdays 10:00-12:00



GRADUATE TAS



Chris Jerrett
Sections 1 & 2
10am - noon



Moxie Broekhuis
Sections 3 & 4
noon - 2pm



Meng Ma
Sections 5 & 6
2pm - 4pm



Zan Li
Sections 7 & 8
4pm - 6pm

UNDERGRADUATE CA / MENTORS



Harika Muthalraj (Course Assistant)



Adrian Hou



Steven Huynh

Ajitesh Bankula



Jacob Hudnut



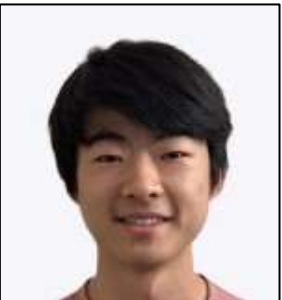
UNDERGRADUATE CA / MENTORS



Ethan Kusse



Braeden Mathers



Jaden Tian

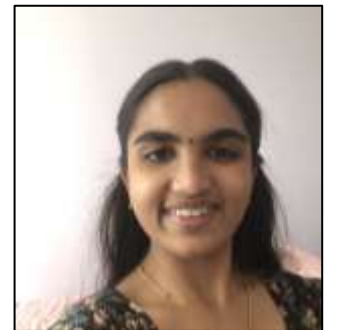
Phoenix Keck



Andre Lungu



Akshaya Ramachandran



COURSE POLICIES



SUBMITTY

- Used for homework submissions / feedback, in-class polling, discussion forum, course announcements
- Discussion forum posts will probably get quicker response than e-mail.
 - Avoid anonymous posts! We want to develop a learning community.
- Do not trust Rainbow Grades!
 - Raw scores are generally OK, but calculated scores, "lowest A", etc. are usually NOT.

IN-CLASS POLLING

- I plan to use Submini (the polling feature of Submittity) to try to collect answers to questions that I pose to the class, and as a means to log attendance.
 - If it doesn't work out, I'll fall back on my old standby of Google Forms.
- As a test right now, please log into Submittity and answer the poll question about interests in CS.
 - [View Poll - Csci2300: Introduction To Algorithms](#)



ATTENDANCE POLICY

- You are expected to be present, on time, and participating in class each day. This includes lectures and labs.
- 25 of the 28 lectures will count towards your grade (so you get three “free” absences). 8 of 9 labs will count towards your grade (so you get one “free” drop/absence).
 - If you know you will be absent, I appreciate a quick e-mail; however, this is not required.
- We will have three exams during the semester. You **must** be present on exam days.

HOMework POLICY

- 11 assignments during semester; drop two lowest scores
 - No new HW the week of exams.
- Homework assigned on Mondays; due in Submittity the following Monday by 8:59pm.
 - Late submissions are not accepted...
 - ... But if you contact me in advance (not the day of) about needing more time, odds are we can work something out.
- Students are encouraged to work together, but you may not directly copy someone else's work – what you submit, you must be able to explain.

LABS

- **No lab this week! Labs start Wed. 1/21.**
 - There also will not be labs on exam weeks.
- Labs are intended to provide a focused, collaborative environment for you to play – and sometimes struggle! – with the concepts presented at a time when guidance and feedback are immediately available.
 - Research shows both that things you have to grind at are more likely to stick with you and that talking it through with others helps!
- You will be expected to stay for the full lab period or until the task is completed. More details next class.

EXAM POLICY

- Three exams during the semester: 2/11, 3/18, 4/15
 - All on Wednesdays from 8-10am in the DCC
 - Open written notes
 - Attendance mandatory; if you're going to miss one of those days you must contact me one week in advance.
 - Lowest of the three will be weighted less.
- Final exam during exam week
 - End of semester travel is NOT an excuse for missing the final!

GRADE CALCULATION

- Lecture attendance: 12.5% ($0.5\% \times 25$ lectures)
- Homework: 22.5% ($2.5\% \times 9$ best assignments)
- Labs: 16% ($2\% \times 8$ best labs)
- In-semester exams: 29% (worst 7%, other two 11% each)
- Final exam: 20%
- Letter conversion: 95+% is required for an A; each step below that is 5% less (so 90% for A-, 85% for B+, ..., down to 50% for a D; below 50% is failing)
 - Hard to get the top mark, easy to get a decent grade, VERY difficult to actually fail

ACADEMIC INTEGRITY

- Any violation of academic integrity is grounds for immediate failure of the course.
 - If you have any doubts about whether something is permitted, ASK FIRST.
 - Never provide another student with your file, for any reason.
 - Specifically disallowed: any sort of AI/LLM tool (e.g. ChatGPT), any sort of "answers" website/service (e.g. Chegg, StackOverflow)
- Please be sure to read the full policy in the syllabus.

THE CARDINAL RULE

If you need help, ask.

- I am usually willing to be flexible on nearly any deadline or other detail, but I can't help if I don't know you need it.
- Worst case, the answer might be "no." No harm done.

SELF CARE

No course is worth damaging your health.

Take care of yourself: eat, sleep, shower, take breaks, exercise, socialize, etc.

Take care of each other: check in, say hello, don't let someone suffer in silence

Don't be afraid to ask for help: We can work around almost any difficulty, but I can't help if I don't know there's an issue.

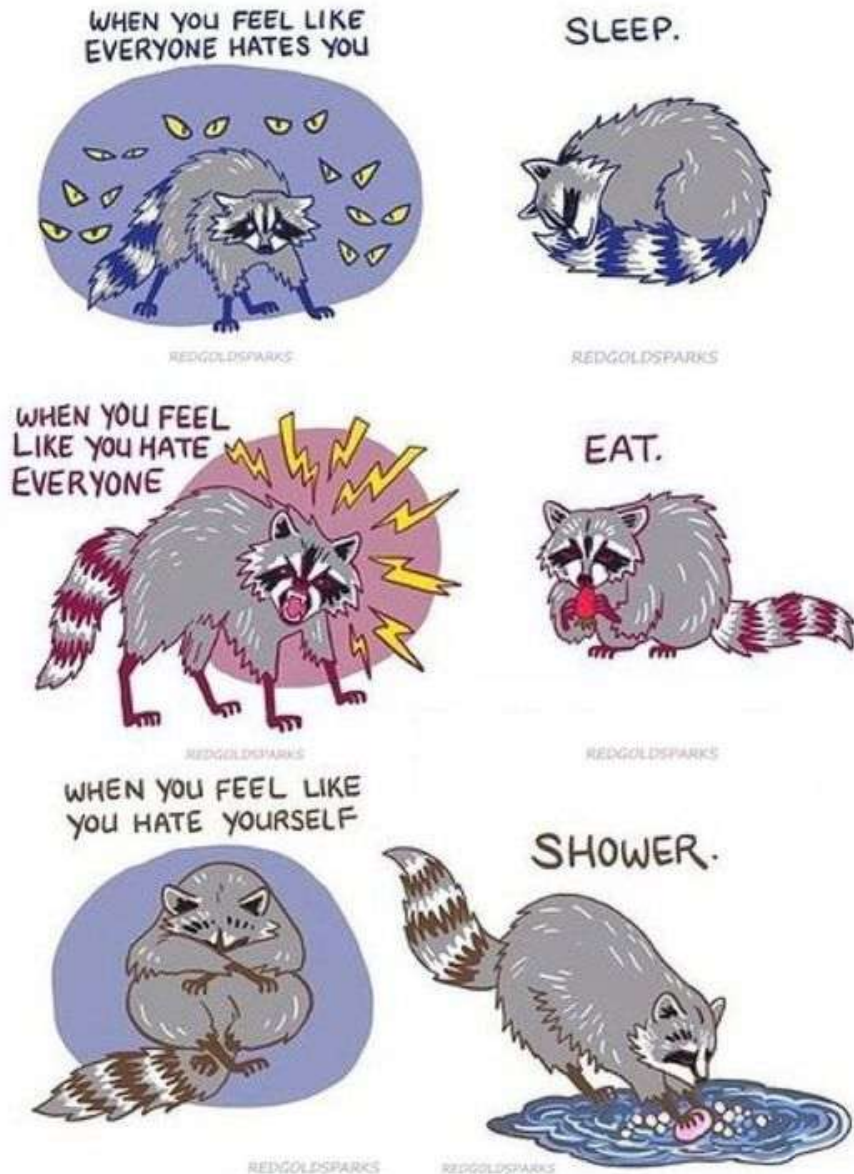
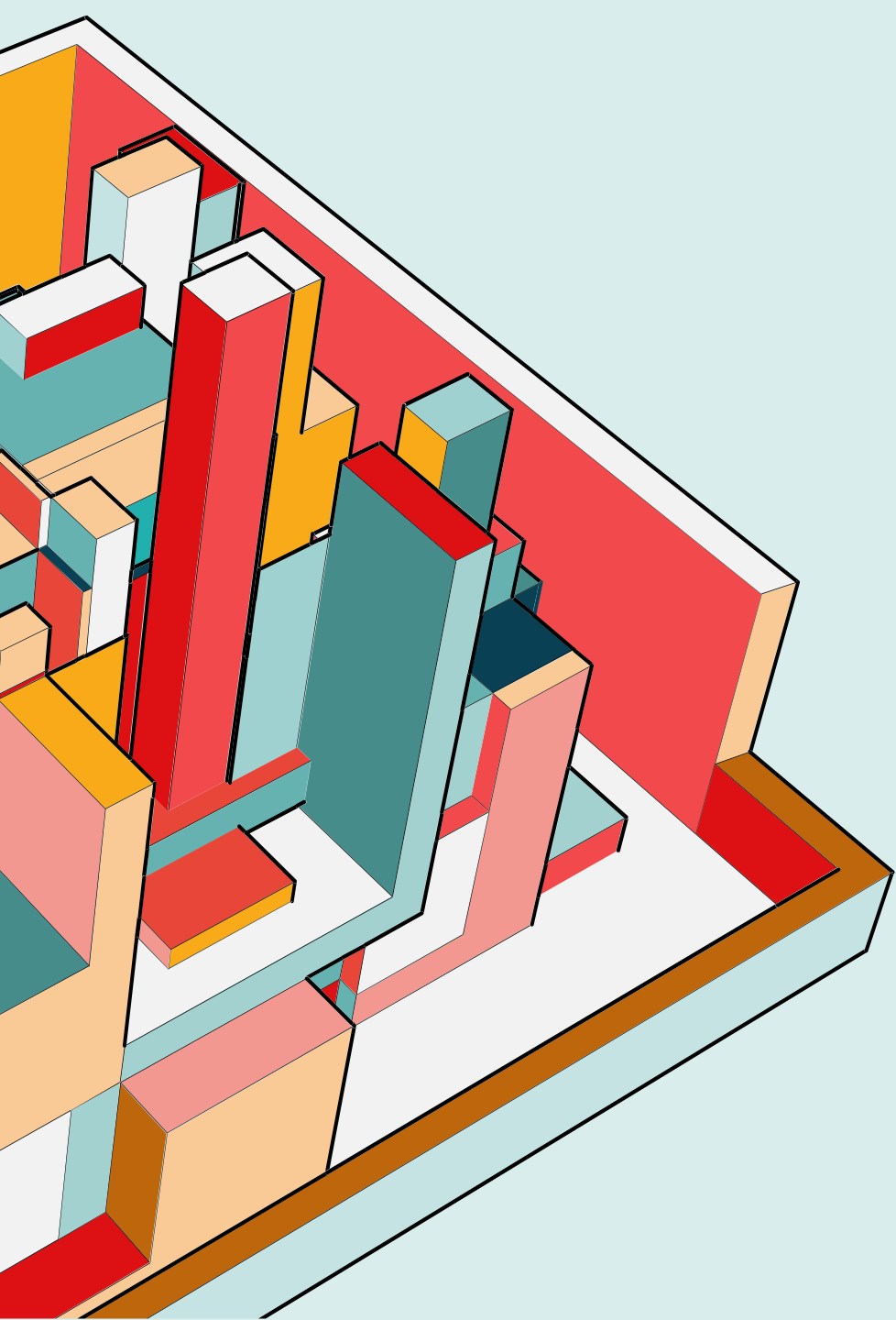


Image credit: Maia Kobabe / @redgoldsparks



INTRODUCTION TO ALGORITHMS

AL-KHWĀRIZMĪ

Muhammad ibn Musa al-Khwārizmī worked at Baghdad's House of Wisdom in the 800s. His most famous work was *al-Kitāb al-Mukhtaṣar fī Ḥisāb al-Jabr wal-Muqābalah* (in English: *The Concise Book of Calculation by Restoration and Balancing*), from which we derive both the word *algebra* as well as its techniques.

Among other works, he also wrote a treatise on using the Hindu numeral system (now known as Arabic numerals!) for arithmetic. The original Arabic text was lost, but Latin copies exist: *Algoritmi de numero Indorum* (*al-Khwārizmī on the Hindu Art of Reckoning*), which gives us the term *algorithm*.



Statue in Khiva, Uzbekistan

WHAT IS AN ALGORITHM?

Take a moment to think about it now.

Now, discuss with your nearby classmates. Are you all on the same page?

There are a variety of ways to word it – I particularly like this definition from Anany Levitin:

“An *algorithm* is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.”

BREAKING DOWN THE DEFINITION

"An *algorithm* is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time."

As we explore our initial example, I urge you to keep in mind the following points:

- The unambiguity of each step cannot be compromised.
- The range of "legitimate inputs" needs to be precisely specified.
- The same algorithm can be represented in multiple ways.
- The same problem may succumb to multiple different algorithms, and their speed and mechanisms may vary greatly.

CONSIDER: GCD

Our initial problem will be greatest common divisor.

Most likely, you first encountered this problem in school during the middle grades. How were you asked to solve it?

A typical set of instructions goes something like this:

- Find all prime factors of the first number.
- Find all prime factors of the second number.
- Multiply all of the common factors together.

CONSIDER: GCD

- Find all prime factors of the first number.
- Find all prime factors of the second number.
- Multiply all of the common factors together.

This is not an algorithm as presented.

- How should we find the prime factors of a number?
- For that matter, how do you find all of the common elements between two lists? (This is not as trivial as you might think...)

The instructions fail to be unambiguous.

CONSIDER: GCD

- Find all prime factors of the first number.
- Find all prime factors of the second number.
- Multiply all of the common factors together.

The instructions fail to be unambiguous.

How might we fix it?

Well, we could specify how to find the prime factors of a number. One approach would be to simply list all of the relevant primes and then test each one.

We could use the Sieve of Eratosthenes to do so.

THE SIEVE OF ERATOSHENES

List all of the integers from 2 up to the target value:

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 ...

Then mark the first one as prime, and eliminate every multiple of that number.

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 ...

Repeat this process until all numbers are prime or eliminated.

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 ...

...

BACK TO GCD

Our algorithm for finding $\gcd(a, b)$ with $a \geq b$ is now:

- Execute the Sieve up to b to generate a list P of primes.
- Try dividing a by each prime in P . If it works, add that prime to the list for a and start back at the beginning of P .
- Repeat this process for b .
- Multiply the common numbers in each list together.

Even this has problems. What are they?

- Finding common numbers. We can skate around this by carefully maintaining P and the lists for a and b in order.

THE BIGGER PROBLEM

What happens if there are no common elements?

We didn't specify... ☹

In fact, the entire problem is underspecified – we didn't say anything about our inputs.

We need to indicate that a and b must be positive integers!

We can also require $a \geq b$ for our own convenience.

The common phrasing in math is “**without loss of generality** (or **WLOG** for short), assume that $a \geq b$ ”. It means that it doesn't actually affect things – we could always begin by comparing and swapping if needed, but that's an implementation detail.

OK, FINE.

Our algorithm for finding $\gcd(a, b)$ with positive integer inputs $a \leq b$ is now:

- Execute the Sieve up to b to generate a list P of primes.
- Try dividing a by each prime in P . If it works, add that prime to the list for a and start back at the beginning of P .
- Repeat this process for b .
- Starting with 1, calculate a running product with each factor common to the lists for a and b , and return the result.

This is, at least, a valid algorithm. Anyone fancy coding it?

THE MIDDLE-SCHOOL ALGORITHM IS _____

Complicated!

... which is why introductory programming classes never have you implement it. Instead, they give you this algorithm:

- Count backwards from b down to 1.
- Try dividing a and b by each number. If they both come out evenly, the answer is that number.

Observe that this is a valid algorithm, since we are guaranteed that 1 will work if nothing else does.

Also, this is super simple to code...

CS1 ALGORITHM - PSEUDOCODE

```
# require: a,b integers with  $a \geq b > 0$ 
def gcd(a,b):
    for i from b down to 1:
        if (a % i == 0) and (b % i == 0):
            return i
```

Of course, some languages would get persnickety because it doesn't look guaranteed that we'll hit that return statement. But this is the glory of pseudocode. 😊

WEAKNESS OF THE CS1 ALGORITHM

The CS1 algorithm is ... not great.

- The runtime is linearly dependent on the value of b .
 - There are factors involving a as well; let's leave that aside for right now.
- It's actually MUCH worse than that. When we talk about the size of the input, we usually mean the number of bits it takes to represent it.
- From that perspective, this algorithm could (very important word there, "could") require an exponential number of iterations.

OLD-SCHOOL GCD

Fortunately, there is a better approach... and it has been known for literally thousands of years. It goes like this:

- Divide a by b and find the remainder.
- If the remainder was 0, the answer is b . Otherwise, replace a by b and b by the remainder, and repeat the process.

```
# require: a,b integers with a >= b > 0
```

```
def gcd(a,b):  
    while(True):  
        r = a % b  
        if (r == 0) return b  
        a = b  
51    b = r
```

OLD-SCHOOL GCD

```
# require: a,b integers with a >= b > 0
def gcd(a,b):
    while(true):
        r = a % b
        if (r == 0) return b
        a = b
        b = r
```

It's not obvious at first that it works at all, let alone that it is faster than the CS1 algorithm – but it does, and it very much is.

In fact, this algorithm is actually linear in the number of bits in the input. Working through the details will be part of HW1.

A FINAL NOTE FROM PERLIS

I think it's extraordinarily important that we in computer science keep **fun** in computing. When it started out, it was an awful lot of fun. [...] after a while we began to take their complaints seriously. **We began to feel as though we really were responsible for the successful, error-free, perfect use of these machines.**

I don't think we are.

I think we're responsible for stretching them, setting them off in new directions, and keeping fun in the house.

Fun comes in many ways. Fun comes in making a discovery, proving a theorem, writing a program, breaking a code. Whatever form or sense it comes in **I hope the field of computer science never loses its sense of fun.**

--from *Structure and Interpretation of Computer Programs*



BEFORE YOU LEAVE

Please take a moment to fill out this anonymous survey about today's lecture.

